

Consultas sobre columnas JSONB

2. Consultas sobre columnas JSONB

Ya sabes guardar datos estructurados en una columna JSONB. El siguiente paso es filtrar por su contenido. Para ello no bastan los comparadores habituales sobre valores escalares: necesitas operadores y funciones de PostgreSQL capaces de acceder a las claves y valores almacenados dentro del JSON.

El problema: consultar dentro de un JSON

Con una columna normal, un WHERE compara el valor completo de esa columna. Con JSONB, lo que quieres comprobar no es la columna entera, sino si existe (o qué vale) una clave concreta dentro del objeto:

```
-- ¿Existe la clave "ebook" de primer nivel en el JSON?
SELECT * FROM libro WHERE jsonb_exists(detalles_edicion, 'ebook');
-- equivalente con el operador ?
SELECT * FROM libro WHERE detalles_edicion ? 'ebook';

-- Extraer el valor de una clave
SELECT detalles_edicion -> 'ebook' FROM libro;      -- como JSON
SELECT detalles_edicion ->> 'ebook' FROM libro;     -- como texto
```

Operadores y funciones JSONB

Operador / función	Qué hace	Devuelve
jsonb_exists(col, clave) / col ? 'clave'	Comprueba si existe una clave de primer nivel — sin importar su valor	boolean
col -> 'clave'	Extrae el valor de una clave, manteniéndolo como JSON (útil para seguir navegando)	jsonb
col ->> 'clave'	Extrae el valor de una clave como texto plano	text

Esto es SQL puro, directamente contra PostgreSQL — antes de verlo envuelto en Java, conviene tenerlo claro a este nivel.

El índice GIN y el operador ?

El índice GIN creado sobre `detalles_edicion` puede acelerar operadores como `?`, que comprueba si existe una clave. Por ejemplo, `WHERE detalles_edicion ? 'ebook'` puede aprovechar el índice cuando el planificador estima que resulta más eficiente que recorrer toda la tabla.

`jsonb_exists(detalles_edicion, 'ebook')` produce el mismo resultado lógico, pero PostgreSQL documenta el operador `?` —no la llamada directa a la función— entre los operadores compatibles con el índice GIN. Por eso no debes dar por hecho que ambas formas utilizarán el índice del mismo modo.

En la actividad de rendimiento se utiliza el operador `?`; en la Specification se utilizará `jsonb_exists(...)` porque Criteria API permite invocar funciones SQL por su nombre.

Un ejemplo completo: disponibleEnFormato

Un filtro más en el buscador — "solo libros disponibles en ebook". El dato vive dentro de la columna JSONB detallesEdicion, no en una columna con su propio tipo simple:

id	titulo	detalles_edicion
1	Cien años de soledad	{"tapaDura":{"..."}, "ebook":{"formato":"epub"}}
2	El nombre del viento	{"bolsillo":{"..."}}
3	Rayuela	{"tapaDura":{"..."}}
4	1984	{"ebook":{"formato":"mobi"}, "bolsillo":{"..."}}

"Solo los disponibles en ebook" debe devolver Cien años de soledad y 1984 —los únicos con la clave "ebook" presente—, y descartar los otros dos.

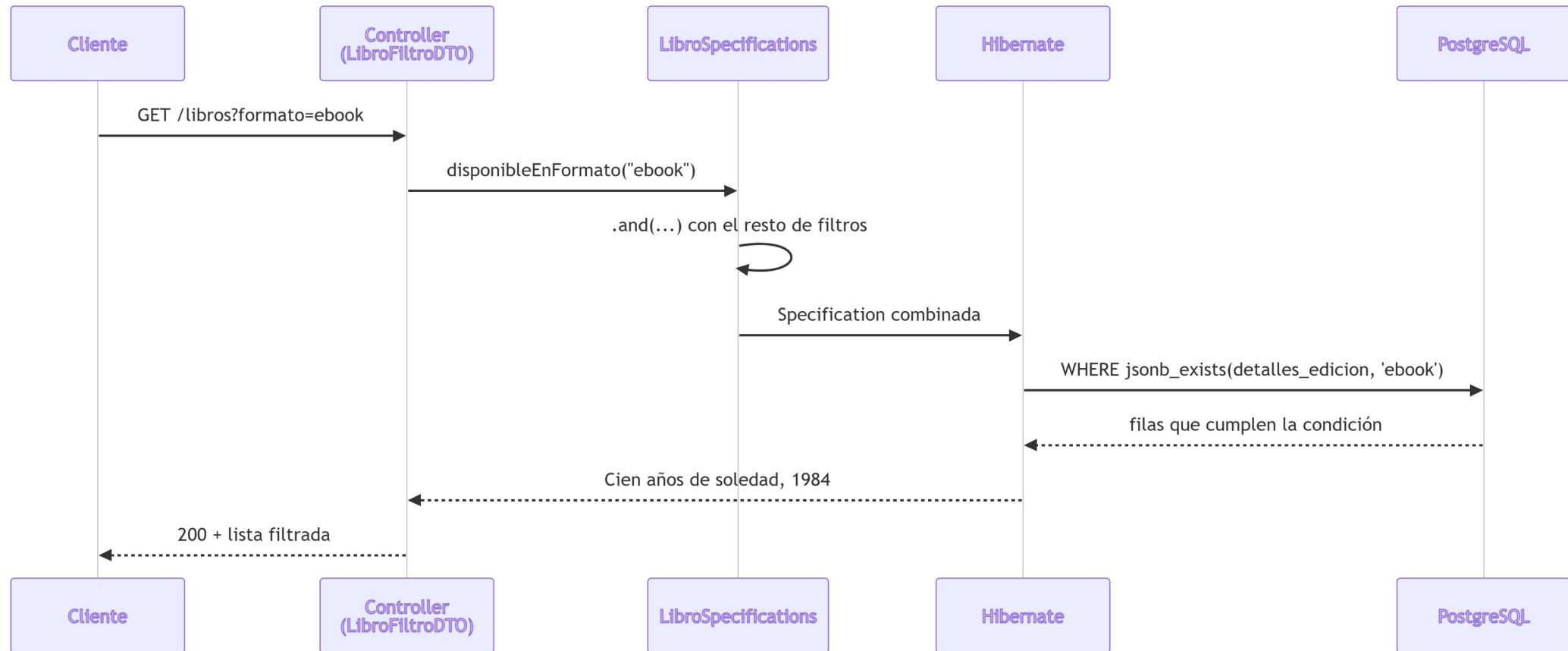
La consulta y su resultado

```
SELECT titulo FROM libro WHERE detalles_edicion = 'ebook';
```

```
titulo
-----
Cien años de soledad
1984
```

Esa es la respuesta que tiene que dar la API cuando alguien busque `?formato=ebook` — pero nadie escribe SQL a mano contra el endpoint: el cliente manda un parámetro, y el código lo traduce a la misma condición `WHERE`.

De la petición HTTP a las dos filas



El parámetro de consulta, el DTO, la Specification, .and(...) ya son conocidos — la novedad es el tramo Hibernate→PostgreSQL: cómo `jsonb_exists(...)` llega a formar parte de la consulta generada.

La Specification, con jsonb_exists

```
public static Specification<Libro> disponibleEnFormato(String formato) {  
    if (formato == null || formato.isBlank()) {  
        return Specification.unrestricted();  
    }  
  
    return (root, query, criteriaBuilder) ->  
        criteriaBuilder.isTrue(  
            criteriaBuilder.function(  
                "jsonb_exists",  
                Boolean.class,  
                root.get("detallesEdicion"),  
                criteriaBuilder.literal(formato)  
            )  
        );  
}
```

Por qué `criteriaBuilder.function(...)`

Compárala con las Specifications "normales" del Tema 1 (`tituloContiene`, `precioMayorOIgualA`), que usan métodos directos de `criteriaBuilder` como `like` o `greaterThanOrEqualTo` — funcionan porque operan sobre una columna con tipo simple propio.

Criteria API es la API de JPA para construir consultas con objetos Java (`CriteriaBuilder`, `Root`, `Predicate...`). Pero no existe un método directo para "existe esta clave en este JSON", porque es una función específica del motor (PostgreSQL), no parte del estándar JPA — por eso hace falta `criteriaBuilder.function(...)`: nombre de la función, tipo de retorno esperado, y sus argumentos.

Combinada con el resto, de forma transparente

```
Specification<Libro> spec = Specification
    .where(LibroSpecifications.tituloContiene(filtro.titulo()))
    .and(LibroSpecifications.precioMayorOIgualA(filtro.precioMin()))
    .and(LibroSpecifications.disponibleEnFormato(filtro.formato()));
```

Filtrar por JSONB o por una columna relacional corriente es exactamente lo mismo desde fuera — una `Specification` más, encadenada con `.and(...)`. Toda la complejidad queda encapsulada dentro de `disponibleEnFormato`.

Y si la clave está más adentro

`jsonb_exists/?` solo comprueban si una clave de primer nivel existe. "¿Tiene ebook?" no es lo mismo que "¿el ebook está en formato mobi?": para esa pregunta más concreta hace falta bajar un nivel:

```
-- Encadenando -> hasta el último nivel, y ->> para el valor final
SELECT titulo FROM libro
WHERE detalles_edicion -> 'ebook' ->> 'formato' = 'mobi';

-- Equivalente con el operador de ruta #>>
SELECT titulo FROM libro
WHERE detalles_edicion #>> '{ebook,formato}' = 'mobi';
```

El resultado ahora es solo 1984 — `jsonb_exists(..., 'ebook')` no distinguiría entre los dos libros, porque nunca mira el valor de la clave, solo si está presente.

Desde Criterias API: jsonb_extract_path_text

```
criteriaBuilder.equal(  
    criteriaBuilder.function(  
        "jsonb_extract_path_text",  
        String.class,  
        root.get("detallesEdicion"),  
        criteriaBuilder.literal("ebook"),  
        criteriaBuilder.literal("formato")  
    ),  
    "mobi"  
);
```

La técnica es idéntica a la de `disponibleEnFormato`: nombre de la función nativa, tipo de retorno, argumentos — solo cambia la función y qué se hace con el resultado.

El índice GIN no acelera esta comparación

El índice GIN sobre la columna completa está pensado para operadores como `?` o `@>`. Una comparación basada en extraer un valor como texto mediante `->>` o `#>>` no aprovecha automáticamente ese mismo índice.

Si una consulta sobre `ebook.formato` se utilizara con mucha frecuencia en una tabla grande, podría crearse un índice de expresión sobre `(detalles_edicion -> 'ebook' ->> 'formato')`. No debe darse por hecho que cualquier consulta sobre JSONB será rápida por tener un índice GIN.

La alternativa: @Query con SQL nativo

Retoma el Tema 1: cuando JPQL no llegaba a algo, la solución era `@Query(nativeQuery = true)`. `jsonb_exists` y `?` son ese mismo caso: no existen en JPQL, porque JPQL solo conoce el vocabulario común de JPA.

```
public interface LibroRepository extends JpaRepository<Libro, Long> {  
  
    @Query(value = "SELECT * FROM libro WHERE jsonb_exists(detalles_edicion, :formato)",  
            nativeQuery = true)  
    List<Libro> buscarPorFormato(@Param("formato") String formato);  
}
```

A diferencia del ranking con `ROW_NUMBER()` del Tema 1 (necesitaba `List<Object[]>`), aquí `SELECT *` selecciona exactamente las columnas de `Libro` — Spring Data mapea el resultado a `List<Libro>` sin problema.

Por qué `jsonb_exists` y no `?` en esta `@Query`

JDBC y JPA utilizan el carácter `?` para representar parámetros posicionales. Dependiendo de la versión y de cómo se procese la consulta nativa, el operador `JSONB ?` puede confundirse con uno de esos marcadores.

Para evitar esa ambigüedad, en estos apuntes se utiliza `jsonb_exists(columna, clave)`, que expresa la misma comprobación de existencia sin emplear el carácter `?`.

La misma alternativa para la ruta anidada

```
@Query(value = "SELECT * FROM libro WHERE detalles_edicion -> 'ebook' ->> 'formato' = :formato",
      nativeQuery = true)
List<Libro> buscarPorFormatoEbook(@Param("formato") String formato);
```

buscarPorFormatoEbook("mobi") devuelve solo 1984 — como aquí no interviene el operador ?, no hay conflicto con el marcador de parámetro de JDBC. ¿Y entonces, criteriaBuilder.function(...) o @Query(nativeQuery = true)? La misma pregunta del Tema 1 entre Specifications y @Query con JPQL, con la misma respuesta:

	Specification + criteriaBuilder.function	@Query(nativeQuery = true)
Se combina con .and(...)	Sí, de forma natural	No — consulta fija, con sus propios parámetros
Sintaxis	Más verbosa (función, tipo, argumentos)	SQL directo, más legible
Cuándo usarla	Un filtro más en un buscador con varias condiciones opcionales	Una consulta propia y fija, pensada solo para esto

Modificar JSONB: reemplazo, no merge

Un update() que recibe un Map nuevo reemplaza el contenido completo de detallesEdicion, no lo combina con el anterior:

	Valor guardado antes	PUT manda	Valor final
Lo que ocurre (reemplazo)	{tapaDura, ebook}	{ebook}	{ebook} — tapaDura desaparece
Lo que se podría esperar (merge, no ocurre)	{tapaDura, ebook}	{ebook}	{tapaDura, ebook} combinados

```
libro.setDetallesEdicion(dto.detallesEdicion()); // sustituye el Map entero
```

Por qué no hay merge parcial en JPA estándar

Para Hibernate, `detallesEdicion` es un único valor de columna — igual que `titulo` o `precio`. JPA no sabe "mirar dentro" de ese valor: trata el Map completo como una unidad atómica que se sustituye entera.

Un merge parcial de verdad exige cargar el objeto, modificar el Map en memoria en Java (añadiendo o quitando claves) y guardar el resultado completo — la combinación ocurre en el código, no en el ORM. `@Transactional` se comporta exactamente igual que sobre cualquier otra entidad.

- `jsonb_exists(columna, clave)` (o el operador `?`) comprueba si una clave existe; `->/->>` extraen valores (como JSON o como texto).
- El índice GIN puede acelerar operadores como `?`. `jsonb_exists(...)` hace la misma comprobación lógica, pero no debe darse por hecho que use el índice igual.
- Para comparar un valor anidado se encadena `->/->>` o se usa el operador de ruta `#>>` — ahí el índice GIN de `?` ya no ayuda igual.
- Consultar JSONB desde Criteria API requiere `criteriaBuilder.function(...)`, porque son funciones específicas del motor, no del estándar JPA.

Ideas clave (cont.)

- `@Query(nativeQuery = true)` es una alternativa más directa cuando la consulta JSONB es fija. Se usa `jsonb_exists(...)` para evitar la ambigüedad entre `?` y los parámetros posicionales de JDBC/JPA.
- Combinada con `.and(...)`, una `Specification` sobre JSONB se usa exactamente igual que una sobre una columna normal — transparente para quien consume el repositorio.
- Actualizar un campo JSONB reemplaza el objeto completo — JPA no sabe hacer merge parcial de un valor de columna; si hace falta, se hace en memoria antes de guardar.
- `@Transactional` no cambia en nada por tener columnas JSONB de por medio.